Εφαρμογή Διαχείρισης Εκδηλώσεων και Ηλεκτρονικών Κρατήσεων στον Παγκόσμιο Ιστό

Ομάδα 42

Βενετσάνου Μαρίνα - 1115202300016

Παπαγεωργίου Στυλιανή - 1115202300157

ΣΤ’ εξάμηνο

Ακαδημαϊκό έτος 2025-2026

Πίνακας Περιεχομένων

Πίνακας Περιεχομένων 2
Πρόλογος 4
Δοκιμαστικοί κωδικοί 5
Τεχνολογίες και Εργαλεία Υλοποίησης 5
Backend 5
Γλώσσα 5
Framework 5
Database 5
Authentication 5
Pillow 5
Django Management Commands 5
SSL/TLS 6
Crontab 6
Frontend 6
Γλώσσα 6
Framework 6
Build Tool 6
Material UI 6
React Router 6
Axios 7
React Leaflet 7
Εγκατάσταση και Εκτέλεση 7
Προαπαιτούμενα 7
Venv 7
Βιβλιοθήκες 8
Database 8
Εκτέλεση Management Commands 8
Εκκίνηση server 8
Οργάνωση Κώδικα 8
Backend 8
Users 9
Events 9
Booking 9
Messaging 9
Exports 9
Certs 9
Logs 9
Recommendation 9
Media 10
Frontend 10
Pages 10
Components 10

Api..... 10
Context..... 10
Utils..... 10
Node_modules.....11
Public..... 11
Index.html..... 11
Package.json..... 11
Vite.config.js..... 11
Λειτουργία Συστήματος.....11
Backend.....11
Models..... 11
Views..... 12
Serializers.....12
Authorization..... 12
Logout..... 12
Frontend..... 13
Components..... 13
Hooks..... 13
Routing..... 14
Επικοινωνία με το API..... 14
Κατάσταση Σύνδεσης..... 14
Έλεγχοι Εγκυρότητας..... 14
Βάση Δεδομένων..... 15
CustomUser..... 15
Category.....15
Event..... 15
EventImage..... 16
EventView..... 16
TicketType..... 16
Booking..... 16
BookingItems.....16
Message.....16
Endpoints.....17
User.....17
Events..... 17
Ticket Type..... 18
Booking..... 18
Messaging..... 19
Exports..... 19
Σχεδιαστικές Επιλογές.....19
Backend..... 19
Αναζήτηση και Φίλτρα..... 19
Βασικό search bar..... 19
Φίλτρα ανά κατηγορία.....20

Φίλτρα Εύρους Τιμών....20
Φίλτρα Χρονικού Διαστήματος....20
Φίλτρα Τοποθεσίας....20
Αυτόματη Ενημέρωση Κατάστασης Εκδηλώσεων....20
Εικόνες....20
Κατάσταση Εκδηλώσεων....20
Συντεταγμένες....20
Χωρητικότητα....21
Περιεχόμενο Κράτησης....21
Pagination....21
Validators....21
Frontend....21
Ενιαία σελίδα δημιουργίας και επεξεργασίας εκδήλωσης....21
Προσωρινή αποθήκευση και δημοσίευση....22
Προκαθορισμένος παραλήπτης στα μηνύματα....22
Ένδειξη νέων μηνυμάτων....22
Επαναχρησιμοποιούμενα components....22
Recommendation System....22
Οργάνωση Κώδικα....22
Offline Dataset....23
Δεδομένα Βάσης....23
Evaluation με βάση metrics....23
Training, Saved Model & Online Recommendation....24
Σχεδιαστικές Επιλογές....24
Επίλογος....24

Πρόλογος

Η εργασία έχει στόχο την ανάπτυξη μιας RESTful εφαρμογής για διαχείριση εκδηλώσεων στο διαδίκτυο. Οι διοργανωτές δημιουργούν και διαχειρίζονται εκδηλωσεις, οι συμμετέχοντες κάνουν κρατήσεις σε εκδηλώσεις και οι διαχειριστές αποδέχονται νέους χρήστες και έχουν τον κύριο έλεγχο της εφαρμογής. Ειδικότερα υπάρχει ένας διαχειριστής από την εγκατάσταση της εφαρμογής που αποδέχεται τις νέες εγγραφές χρηστών. Παράλληλα προτείνονται εξατομικευμένες εκδηλώσεις για κάθε χρήστη ξεχωριστά. Παρακάτω παρουσιάζονται τα εξής:

- **τεχνολογίες και εργαλεία,**
- **οδηγίες εγκατάστασης και εκτέλεσης,**
- **οργάνωση του κώδικα,**
- **λειτουργία συστήματος,**
- **βάση δεδομένων,**
- **endpoints,**
- **σχεδιαστικές επιλογές,**
- **recommendation system,**
- **επίλογος με τις προκλήσεις-δυσκολίες που αντιμετωπίσαμε κατά την υλοποίηση.**

Δοκιμαστικοί κωδικοί

Διοργανωτής: organizer Organizer12@
Επισκέπτης: attendee Attendee123@
Διαχειριστής: admin admin@

Τεχνολογίες και Εργαλεία Υλοποίησης

Backend

Γλώσσα

Υλοποιήθηκε σε γλώσσα προγραμματισμού Python.

Framework

Το Django χρησιμοποιήθηκε ως το βασικό framework της εφαρμογής. Για την ανάπτυξη του REST API χρησιμοποιήθηκε το **Django REST Framework (DRF)**, το οποίο παρέχει serializers για μετατροπή python objects σε JSON και αντίστροφα, generic views για την υλοποίηση CRUD λειτουργιών, permissions system και εύκολο authentication.

Database

Χρησιμοποιήθηκε η default βάση δεδομένων του DRF η **SQLite**.

Authentication

Χρησιμοποιήθηκε ο μηχανισμός **JSON Web Token (JWT)** μέσω της βιβλιοθήκης **Simple JWT** του DRF. Μετά το login ο χρήστης λαμβάνει και το access token, το οποίο αποστέλλεται σε κάθε request. Μετά την αυθεντικοποίηση τίθεται το αντικείμενο request.user που είναι ο τρέχων συνδεδεμένος χρήστης.

Pillow

Η βιβλιοθήκη pillow χρησιμοποιήθηκε για την αποθήκευση εικόνων των εκδηλώσεων. Συγκεκριμένα χρησιμοποιήθηκε το ImageField του Django για αποθήκευση και διαχείριση εικόνων, το οποίο απαιτεί την pillow ώστε να επικυρώνει και να επεξεργάζεται τις εικόνες. (source: https://medium.com/@prashantmaharjan5/complete-guide-to-image-uploads-in-django-4f24e0422708)

Django Management Commands

Για αυτοματοποιημένες εργασίες έγινε χρήση Django Management Commands. Ειδικότερα για τον περιοδικό έλεγχο του endDateTime όλων των εκδηλώσεων ώστε να αλλάζει το status του σε COMPLETED και να μην εμφανίζονται στις ενεργές εκδηλώσεις, όπως και για την

εγκατάσταση ενός admin (python manage.py create_admin). Αν υπάρχει ήδη δεν τον εγκαθιστά ξανά.

SSL/TLS

Δημιουργήθηκε self-signed certificate με την εντολή, με οδηγίες από: https://letsencrypt.org/docs/certificates-for-localhost/

Crontab

Για την αυτοματοποίηση κάποιων εργασιών χρησιμοποιήθηκαν management commands που τρέχουν ως cron jobs. Συγκεκριμένα η ενημέρωση της κατάστασης των εκδηλώσεων σε COMPLETED γίνεται μια φορά την ημέρα και η εκπαίδευση του BMF μοντέλου για τον αλγόριθμο συστάσεων, κάθε μισή ώρα. Φυσικά οι εντολές τρέχουν και χειροκίνητα με

```
python manage.py <command>
```

Frontend

Γλώσσα

Χρησιμοποιήθηκε η Javascript, με χρήση της σύνταξης jsx για την δήλωση των στοιχείων της διεπαφής

Framework

Χρησιμοποιήθηκε η βιβλιοθηκη React.

Build Tool

Το Vite χρησιμοποιήθηκε ως build tool και development server, γιατί έχει γρήγορο περιβάλλον ανάπτυξης

Material UI

Χρησιμοποιήθηκε η βιβλιοθήκη MUI που παρέχει έτοιμα components και το sx prop, που επιτρέπει τον ορισμό στυλ απευθείας στο component χωρίς ξεχωριστά αρχεία css

React Router

Χρησιμοποιήθηκε το React Router για την πλοήγηση. Η εφαρμογή λειτουργεί ως single page application: η αλλαγή σελίδας γίνεται στον browser, χωρίς νέο αίτημα στον server. Υποστηρίζει δυναμικές διαδρομές (π.χ. /events/:id) και μεταφορά δεδομένων μεταξύ σελίδων μέσω του location state.

Axios

Χρησιμοποιήθηκε το Axios για τις κλήσεις προς το REST API. Επιλέχθηκε έναντι του fetch κυρίως για τους interceptors, που επιτρέπουν την αυτόματη προσθήκη του access token σε κάθε αίτημα και τη διαχείριση της λήξης του.

React Leaflet

Χρησιμοποιήθηκε για τον χάρτη OpenStreetMap

SSL/TLS
Ο development server του Vite ρυθμίστηκε ώστε να εξυπηρετεί μέσω HTTPS, με χρήση των ίδιων self signed certificates που χρησιμοποιεί και το backend. Η ρύθμιση γίνεται στο αρχείο vite.config.js

Εγκατάσταση και Εκτέλεση

Προαπαιτούμενα

- Python 3.12 ή νεότερη
- pip
- Git
- Virtual Environment (venv)

Εγκατάσταση Repo

```
git clone git@github.com:venetsanoum/event-booking-app.git
cd backend
```

Venv

Δημιουργία:

```
python -m venv venv
```

Ενεργοποίηση για Linux:

```
source venv/bin/activate
```

Αντίστοιχα γιια να τρέξει το frontend, την πρώτη φορα:

```
npm install
```

Βιβλιοθήκες

Εγκαθίστανται μέσω του requirements.txt

```
pip install -r requirements.txt
```

Database

```
python manage.py makemigrations
python manage.py migrate
```

Εκτέλεση Management Commands

Πριν την εκκίνηση της εφαρμογής πρέπει να τρέξουν τα custom managements commands.

Για την ενημέρωση της κατάστασης των εκδηλώσεων σε COMPLETED:

```
python manage.py stats
```

και για την εγκατάσταση του admin (μία φορά)

```
python manage.py create_admin
```

Εκκίνηση server

μέσα στον φάκελο backend

```
python manage.py runserver_plus --cert-file certs/localhost.crt --key-file certs/localhost.key
```

μέσα στον φάκελο frontend

```
npm run dev
```

Το app θα είναι διαθέσιμο στην τοπική διεύθυνση: https://localhost:5173/

Οργάνωση Κώδικα

Backend

-backend/
- -backend/
- -booking/
- -certs/
- -events/

-management/
-exports/
-logs/
-media/
-messaging/
-recommendation/
-users/
-management/
-manage.py
-requirements.txt

Users

Διαχείριση χρηστών, authentication, permissions. Το subdirectory management/ περιέχει custom management command, το οποίο πρέπει να τρέξει κάποιος πριν την έναρξη του server για μια φορα, ώστε να δημιουργηθεί ένας admin user.

Events

Διαχείριση εκδηλώσεων, τύπων εισιτηρίων, εικόνων και λειτουργίας αναζήτησης. Το subdirectory management/ περιέχει custom management command, ώστε να ανανεώνεται το status των εκδηλώσεων από PUBLISHED σε COMPLETED όταν περάσει η ημερομηνία λήξης τους.

Booking

Διαχείριση κρατήσεων

Messaging

Ανταλλαγή μηνυμάτων μεταξύ διοργανωτών και συμμετεχόντων

Exports

Για τη δυνατότητα εξαγωγής όλων των εκδηλώσεων του συστήματος που έχει ο admin.

Certs

Περιέχει το self signed certificate στο localhost.crt και το αντίστοιχο κλειδί.

Logs

Γράφονται οι εκτελέσεις των management commands που τρέχουν με cron.

Recommendation

Η υλοποίηση του αλγορίθμου συστάσεων.

Media

Εκεί αποθηκεύονται όλες οι εικόνες που χρησιμοποιούνται και ανεβαίνουν στην εφαρμογή από τους χρήστες.

Frontend

-frontend/
- -node_modules/
- -public/
- -src/
  - -api/
  - -components/
  - -context/
  - -pages/
  - -utils/
- -eslint.config.js
- -index.html
- -package-lock.json
- -package.json
- -vite.config.js

Pages

Μια σελίδα ανά διαδρομή της εφαρμογής. Κάθε σελίδα αναλαμβάνει την φόρτωση των δεδομένων της και τη διαχείριση της δικής της κατάστασης.

Components

Στοιχεία που επαναχρησιμοποιούνται σε πολλές σελίδες.

Api

Οι κλήσεις προς το backend, ομαδοποιημένες ανά ενότητα. Περιλαμβάνει και το κοινό instance του axios με τους interceptors.

Context

Η κατάσταση σύνδεσης του χρήστη.

Utils

Βοηθητικές συναρτήσεις, όπως η μορφοποίηση ημερομηνιών

Node_modules

Οι εγκατεστημένες βιβλιοθήκες που δημιουργούνται αυτόματα με το npm install

Public

Στατικά στοιχεία που έχουμε όπως η εικόνα φόντου και το εικονίδιο της καρτέλας

Index.html

Το μοναδικό αρχείο html, μέσα στο οποίο φορτώνει η react.

Package.json

Οι εξαρτήσεις του project και οι διαθέσιμες εντολές. Το package-lock.json καταγράφει τις ακριβείς εκδόσεις, ώστε η εγκατάσταση να είναι πανομοιότυπη σε κάθε μηχάνημα

Vite.config.js

Ρυθμίσεις του development server, συμπεριλαμβανομένης της λειτουργίας μέσω HTTPS

Λειτουργία Συστήματος

Backend

Το DRF ακολουθεί την αρχιτεκτονική Model-View-Serializer. Η ροή γενικά είναι η εξής: ο client επικοινωνεί με τον server μέσω https αιτημάτων. Ο server επεξεργάζεται τα ερωτήματα, κάνει ελέγχους εγκυρότητας και δικαιωμάτων, επικοινωνεί με τη βάση δεδομένων και επιστρέφει ένα response σε JSON. Ειδικότερα:

Models

Εδω περιλαμβάνεται η αναπαράσταση των δεδομένων και τον τρόπο αποθήκευσης τους στη βάση. Τα models της εφαρμογής είναι:

- CustomUser
- Event
- TicketType
- Booking
- BookingItems
- Message
- Category
- EventImage

- EventView

τα οποία συνδέονται μέσω ForeignKey και ManyToMany σχέσεις

Views

Αποτελούν την είσοδο κάθε request. Μέσα από το url θα κληθεί το αντίστοιχο view, το οποίο:

- θα παραλάβει το request,
- θα επιλέξει κατάλληλο serializer,
- θα κάνει έλεγχο των permissions,
- θα επιτρέψει το κατάλληλο response

Στην εφαρμογή έγινε χρήση κυρίως των Generic Views του DRF, τα οποία εκτελούν εργασίες χωρίς να απαιτείται πολύ γράψιμο κώδικα. Ειδικότερα:

- CreateAPIView
- ListAPIView
- RetrieveAPIView
- UpdateAPIView
- DestroyAPIView
- RetrieveUpdateDestroyAPIView

Serializers

Μετατρέπουν τα αντικείμενα της βάσης σε JSON, μετατρέπουν JSON σε python objects, κάνουν validate τα δεδομένα. Στην εφαρμογή κάνουν ελέγχους για:

- χωρητικότητα εκδηλώσεων,
- διαθεσιμότητα εισιτηρίων,
- ημερομηνίες έναρξης-λήξης,
- αλλαγή εισιτηρίων κλπ

Authorization

Μετά την αυθεντικοποίηση σε JWT, εφαρμόζεται έλεγχος δικαιωμάτων μέσω custom permission classes. Υπάρχουν permissions όπως:

- IsOrganizer,
- IsEventOrganizer : για να ελέγχει αν ο χρήστης είναι οργανωτής ενός συγκεκριμένου event,
- isAppAdmin,
- isDraftedEvent,
- isEventOrganizerTicketEdition : ελέγχει αν ο χρήστης είναι οργανωτής ενός συγκεκριμένου event, με τη διαφορά ότι στα συγκεκριμένα endpoints που γίνεται αυτός ο έλεγχος το primary key του url είναι το ticket_id και όχι το event_id.

Logout

Κατά το longout το endpoint δέχεται στο request body το refresh token και το κάνει blacklist. Εφόσον είναι blacklisted δεν μπορεί να χρησιμοποιηθεί για την ανάκτηση νέου access token. Έτσι το refresh token ακυρώνεται πριν την προκαθορισμένη λήξη του

Frontend

Components

Η διεπαφή είναι χτισμένη από components, δηλαδή συναρτήσεις που επιστρέφουν το κομμάτι της οθόνης που τους αντιστοιχεί. Ένα component μπορεί να περιέχει άλλα, ώστε η σελίδα να συντίθεται από μικρότερα, ανεξάρτητα μέρη.

Τα δεδομένα μεταφέρονται από το εξωτερικό component προς τα εσωτερικά μέσω props. Για παράδειγμα, η σελίδα αναζήτησης φέρνει τη λίστα των εκδηλώσεων και δίνει σε κάθε κάρτα τα στοιχεία μιας εκδήλωσης.

Στην εφαρμογή διακρίνονται σε δύο κατηγορίες:

- **Σελίδες**, που αντιστοιχούν σε μία διαδρομή του URL και αναλαμβάνουν να φέρουν τα δεδομένα τους από το API
- **Επαναχρησιμοποιούμενα components**, όπως η μπάρα πλοήγησης, οι κάρτες εκδηλώσεων, τα κουμπιά και τα breadcrumbs, που εμφανίζονται σε πολλές σελίδες και δέχονται από αυτές ό,τι χρειάζονται να δείξουν

Hooks

Τα hooks είναι έτοιμες συναρτήσεις της React που δίνουν στα components δυνατότητες πέρα από την απλή εμφάνιση. Χρησιμοποιήθηκαν:

- **useState**: αποθηκεύει τιμές που αλλάζουν, όπως τα περιεχόμενα μιας φόρμας ή τα δεδομένα που ήρθαν από το API. Κάθε φορά που αλλάζει μια τέτοια τιμή, η σελίδα ενημερώνεται αυτόματα.
- **useEffect**: εκτελεί κώδικα όταν φορτώνει η σελίδα ή όταν αλλάζει κάποια συγκεκριμένη τιμή. Χρησιμοποιείται κυρίως για να ζητηθούν δεδομένα από το API.
- **use Params**: διαβάζει το μεταβλητό κομμάτι της διεύθυνσης, για παράδειγμα τον αριθμό της εκδήλωσης στο `/events/5`
- **useNavigate**: μεταφέρει τον χρήστη σε άλλη σελίδα μέσα από τον κώδικα, για παράδειγμα μετά την ολοκλήρωση μιας κράτησης.
- **useLocation**: δίνει πρόσβαση στην τρέχουσα διεύθυνση και στα δεδομένα που στέλνονται από την προηγούμενη σελίδα.
- **useContext**: διαβάζει τα στοιχεία του συνδεδεμένου χρήστη, που είναι κοινά για όλη την εφαρμογή.

Routing

Το αρχείο App.jsx ορίζει τις διαδρομές της εφαρμογής, αντιστοιχίζοντας κάθε url σε μία σελίδα. Υποστηρίζονται δυναμικές διαδρομές, όπου τμήμα του url αποτελεί παράμετρο, όπως /events/:id. Μέσω του location state μεταφέρονται δεδομένα μεταξύ σελίδων χωρίς να εμφανίζονται στο url, όπως τα επιλεγμένα εισιτήρια κατα την μετάβαση στην επιβεβαίωση κράτησης.

Επικοινωνία με το API

Όλες οι κλήσεις γίνονται μέσω κοινού instance του axios, στο οποίο ορίζονται δύο interceptors:

- Ο interceptor αιτήματος προσθέτει αυτόματα το access token σε κάθε κλήση, εφόσον υπάρχει. Ο έλεγχος ύπαρξης είναι απαραίτητος, ώστε τα δημόσια endpoints να είναι προσβάσιμα και από μη συνδεδεμένους χρήστες.
- Ο interceptor απόκρισης αναγνωρίζει τη λήξη του access token από την απόκριση 401, ζητά νέο μέσω του refresh token και επαναλαμβάνει αυτόματα το αρχικό αίτημα. Αν δεν υπάρχει refresh token ή η ανανέωση αποτύχει, τα στοιχεία σύνδεσης διαγράφονται και ο χρήστης οδηγείται στο login

Κατάσταση Σύνδεσης

Τα στοιχεία του συνδεδεμένου χρήστη διατηρούνται σε React Context, ώστε να είναι διαθέσιμα σε οποιοδήποτε σημείο της εφαρμογής χωρίς μεταφορά props μέσω ενδιάμεσων components. Παράλληλα αποθηκεύονται στο localStorage μαζί με τα tokens, ώστε η σύνδεση να διατηρείται μετά από ανανέωση της σελίδας.

Έλεγχοι Εγκυρότητας

Στις φόρμες εφαρμόζονται έλεγχοι πριν την αποστολή, όπως η πληρότητα των υποχρεωτικών πεδίων κατά τη δημοσίευση εκδήλωσης, η ταύτιση των κωδικών, και η μη μείωση της ποσότητας εισιτηρίων κάτω από τα ήδη πωλημένα. Οι έλεγχοι αυτοί λειτουργούν συμπληρωματικά προς εκείνους του backend, με σκοπό την άμεση ανατροφοδότηση του χρήστη και όχι την αντικατάστασή τους.

Βάση Δεδομένων

CustomUser

Κάνει extend τον AbstractUser του DRF, και έχει συνολικά τα εξής πεδία:

- **username,**
- **email,**
- **password,**
- **first_name,**
- **last_name,**
- **phone,**
- **address,**
- **city,**
- **country,**
- **afm,**
- **latitude, longitude**: πληροφορίες τοποθεσίες
- **role** (ADMIN, ORGANIZER, ATTENDEE)
- **approval_status** (PENDING, ACCEPTED, REJECTED).

Category

Για την κατηγοριοποίηση εκδηλώσεων, έχει πεδίο name

Event

Τα πεδία του ειναι τα εξής:

- **title**: τίτλος της εκδήλωσης.
- **organizer**: ο διοργανωτής της εκδήλωσης, ο οποίος συνδέεται με το μοντέλο Custom User με σχέση 1-N (ένας οργανωτής έχει πολλές εκδηλώσεις)
- **category**: οι κατηγορίες στις οποίες ανήκει η εκδήλωση (εδώ έχουμε σχέση many to many)
- **eventType**: ο τύπος της εκδήλωσης.
- **venue**: ο χώρος διεξαγωγής.
- **address**, **city**, **country**: στοιχεία τοποθεσίας.
- **latitude** και **longitude**: γεωγραφικές συντεταγμένες της εκδήλωσης.
- **startDateTime** και **endDateTime**: ημερομηνία και ώρα έναρξης και λήξης.
- **capacity**: η μέγιστη χωρητικότητα της εκδήλωσης.
- **description**: αναλυτική περιγραφή.
- **status**: η τρέχουσα κατάσταση της εκδήλωσης (draft, published, cancelled, completed)

EventImage

Έχει πεδίο ImageField, το οποίο αποθηκεύει τις εικόνες στον φάκελο events/ και το Event (σχέση 1-N, ένα event έχει πολλές εικόνες).

EventView

Το μοντέλο της βάσης που κρατάει το ιστορικό των επισκέψεων των χρηστών σε σελίδες event. Χρησιμοποιείται για το recommendation system.

TicketType

Περιγράφει τους διαθέσιμους τύπους εισιτηρίων μιας εκδήλωσης. Πεδία:

- **name**: ονομασία του τύπου εισιτηρίου (π.χ. General Admission, VIP, Student).
- **price**: τιμή αγοράς του εισιτηρίου.
- **quantity**: συνολικός αριθμός εισιτηρίων του συγκεκριμένου τύπου.
- **available**: αριθμός εισιτηρίων που παραμένουν διαθέσιμα προς πώληση.
- **event**: η εκδήλωση στην οποία ανήκει ο συγκεκριμένος τύπος εισιτηρίου (1-N σχέση)

Booking

Αναπαριστά μια κράτηση. Έχει τα εξής πεδία:

- **attendee**: ο χρήστης που πραγματοποίησε την κράτηση (1-N σχέση, ένας χρήστης μπορεί να έχει πολλές κρατήσεις στο όνομά του).
- **time**: ημερομηνία και ώρα δημιουργίας της κράτησης.
- **cost**: το συνολικό κόστος της κράτησης.
- **event**: η εκδήλωση για την οποία πραγματοποιήθηκε η κράτηση (1-N σχέση, ένα event έχει πολλές κρατήσεις).

BookingItems

Αποθήκευση εισιτηρίων. Χρησιμοποιήθηκε για να επιτρέψει σε μια κράτηση να έχει παραπάνω από έναν τύπο εισιτηρίων. Πεδία:

- **booking**: την κράτηση στην οποία ανήκει (1-N, μια κράτηση μπορεί να περιλαμβάνει περισσότερα από ένα bookingItems).
- **ticket_type**: τον τύπο εισιτηρίου που επιλέχθηκε (1-N, μια κράτηση μπορεί να περιλαμβάνει περισσότερους από έναν τύπο εισιτηρίων).
- **number**: το πλήθος εισιτηρίων του συγκεκριμένου τύπου.

Message

Αναπαριστά ένα μήνυμα στην εφαρμογή. Πεδία:

- **sender**: ο αποστολές (1-N σχέση, μιας και κάποιος στέλνει πολλά μηνύματα).
- **receiver**: ο παραλήπτης (1-N σχέση, κάποιος λαμβάνει πολλά μηνύματα).

- **event**: το event με βάση το οποίο ξεκίνησε η συνομιλία (1-N σχέση, σε ένα event αντιστοιχουν πολλά μηνύματα).
- **content**: περιεχόμενο
- **subject**: θέμα μηνύματος
- **sendDateTime**: ώρα αποστολής.
- **isRead**: αν έχει διαβαστεί απο τον παραλήπτη.
- **isDeletedBySender**: αν έχει διαγραφεί από τον αποστολέα δηλαδή να μην εμφανίζεται στα sent του.
- **isDeletedByReceiver**: αν έχει διαγραφεί από τον παραλήπτη δηλαδή να μην εμφανίζεται στο inbox του.

Σημείωση: *η διαγραφή εδώ δεν αφορά διαγραφή από την βάση, απλά "εξαφάνιση" του μηνύματος από τον αντίστοιχο κατάλογο εισερχομένων/απεσταλμένων, μιας και θέλουμε ενδεχομένως να είναι διαθέσιμο στον άλλον χρήστη ο οποίος πιθανόν να μην έχει διαγράψει το μήνυμα από τον δικό του κατάλογο.*

Endpoints

User

- **POST /users/register/**: Εγγραφή νέου χρήστη.
- **POST /users/login/**: Σύνδεση χρήστη και έκδοση JWT tokens.
- **POST /users/logout/**: Αποσύνδεση χρήστη, γίνεται blacklisted το refresh token, ώστε να μην μπορεί να χρησιμοποιηθεί -μετά το logout και για όσο είναι ενεργό- για την απόκτηση access token.
- **GET /users/profile/**: Προβολή των στοιχείων του συνδεδεμένου χρήστη.
- **GET /admin/users/**: Προβολή όλων των χρηστών της εφαρμογής (μόνο διαχειριστής).
- **GET /admin/users/{id}/**: Προβολή στοιχείων συγκεκριμένου χρήστη (μόνο διαχειριστής).
- **PATCH /admin/users/{id}/approval/**: Έγκριση ή απόρριψη αίτησης εγγραφής διοργανωτή (μόνο διαχειριστής).
- **GET /admin/users/pending/**: Προβολή όλων των εκκρεμών αιτήσεων έγκρισης διοργανωτών.

Events

- **POST /events/**: Δημιουργία νέας εκδήλωσης. Ουσιαστικά εδώ θα αποθηκευτεί ως DRAFT, ώστε μετά με διαδοχικές κλήσεις των endpoint **POST /events/{id}/ticket-types** και **POST /events/{id}/images** να καταλήξει στο **POST events/{id}/publish/** όπου θα ανέβει η τελική εκδήλωση.
- **POST /events/draft-events/**: Δημιουργία draft εκδήλωσης. Ουσιαστικά κάνει τη προσωρινή αποθήκευση δηλαδή βάζει status=DRAFT.
- **POST events/{id}/publish/**: Τελικό ανέβασμα draft εκδήλωσης

- **POST /events/{id}/ticket-types**: Δημιουργία νέου τύπου εισιτηρίου για συγκεκριμένη εκδήλωση.
- **POST /events/{id}/images**: Προσθήκη εικόνας σε συγκεκριμένη εκδήλωση.
- POST /events/{id}/publish/: Είναι το endpoint για το ανέβασμα εκδήλωσης που είναι draft
- **GET /events/my-events**: Προβολή όλων των εκδηλώσεων του διοργανωτή.
- **GET /events/my-drafted-events**: Προβολή εκδηλώσεων που βρίσκονται σε κατάσταση Draft.
- **GET /events/my-cancelled-events**: Προβολή ακυρωμένων εκδηλώσεων.
- **GET /events/my-completed-events**: Προβολή ολοκληρωμένων εκδηλώσεων.
- **GET /events/{id}/preview/**: Προβολή εκδηλώσεων από τον διοργανωτή που είναι CANCELLED, COMPLETED, PUBLISHED ώστε να βλέπει στοιχεία για κρατήσεις και εναπομείναντα εισιτήρια.
- **PUT /events/{id}/edit**: Πλήρης ενημέρωση στοιχείων εκδήλωσης.
- **PATCH /events/{id}/edit**: Μερική ενημέρωση στοιχείων εκδήλωσης.
- **DELETE /events/{id}/edit**: Διαγραφή εκδήλωσης.
- **GET /events/search**: Αναζήτηση εκδηλώσεων με χρήση φίλτρων.
- **POST /events/category**: Δημιουργία νέας κατηγορίας εκδηλώσεων (μόνο διαχειριστής).
- **GET /events/category-list**: Προβολή όλων των διαθέσιμων κατηγοριών.
- **GET /events/all**: Προβολή όλων των εκδηλώσεων ανεξαρτήτως κατάστασης.
- **GET /events/active**: Προβολή όλων των δημοσιευμένων (ενεργών) εκδηλώσεων.
- **GET /events/details/{id}**: Προβολή αναλυτικών πληροφοριών συγκεκριμένης εκδήλωσης.
- **GET /events/cities**: Προβολή όλων των πόλεων στις οποίες υπάρχουν ενεργές εκδηλώσεις.

Ticket Type

- **GET /events/all-ticket-types**: Προβολή όλων των τύπων εισιτηρίων.
- **GET /events/{id}/event-ticket-types**: Προβολή των τύπων εισιτηρίων συγκεκριμένης εκδήλωσης.
- **PATCH /events/{event_id}/ticket-types/{ticket_type_id}**: Τροποποίηση στοιχείων τύπου εισιτηρίου (όνομα, τιμή, ποσότητα).
- **DELETE /events/{event_id}/delete-ticket-types/{ticket_type_id}**: Διαγραφή τύπου εισιτηρίου συγκεκριμένης εκδήλωσης.

Booking

- **POST /booking/create/{event_id}**: Δημιουργία νέας κράτησης για συγκεκριμένη εκδήλωση.
- **GET /booking/my-bookings**: Προβολή όλων των κρατήσεων του συνδεδεμένου χρήστη.
- **GET /booking/my-cancelled-bookings**: Προβολή ακυρωμένων κρατήσεων.
- **GET /booking/my-upcoming-bookings**: Προβολή επερχόμενων κρατήσεων.
- **GET /booking/my-completed-bookings**: Προβολή ολοκληρωμένων κρατήσεων.

- **GET /booking/{booking_id}**: Προβολή αναλυτικών πληροφοριών συγκεκριμένης κράτησης.
- **GET /booking/event/{event_id}**: Προβολή όλων των κρατήσεων συγκεκριμένης εκδήλωσης (για τον διοργανωτή).
- **PATCH /booking/<int:event_id>/cancel/<int:booking_id>/**: Ακύρωση κράτησης από τον διοργανωτή.

Messaging

- **POST /messages/{event_id}**: Αποστολή νέου μηνύματος σχετικά με συγκεκριμένη εκδήλωση.
- **GET /messages/inbox**: Προβολή των εισερχόμενων μηνυμάτων.
- **GET /messages/sent**: Προβολή των απεσταλμένων μηνυμάτων.
- **GET /messages/unread**: Προβολή των μη αναγνωσμένων μηνυμάτων.
- **GET /messages/unread-count**: Επιστροφή του συνολικού αριθμού των μη αναγνωσμένων μηνυμάτων.
- **GET /messages/{id}**: Προβολή συγκεκριμένου μηνύματος.
- **PATCH /messages/inbox/{id}**: Απόκρυψη μηνύματος από τα εισερχόμενα του χρήστη.
- **PATCH /messages/sent/{id}**: Απόκρυψη μηνύματος από τα απεσταλμένα του χρήστη.

Exports

- **GET /exports/xml**: ο admin μπορεί να κατεβάζει xml αρχείο για όλες τις εκδηλώσεις της εφαρμογής.
- **GET /exports/json**: ο admin μπορεί να κατεβάζει jsonαρχείο για όλες τις εκδηλώσεις της εφαρμογής.

Σχεδιαστικές Επιλογές

Backend

Αναζήτηση και Φίλτρα

Για την αναζήτηση χρησιμοποιήθηκε η βιβλιοθήκη django-filter. Τα φίλτρα υλοποιήθηκαν μέσω της κλάσης EventFilter, που κάνει extend την κλάση FilterSet του django-filter.

Βασικό search bar

Υπάρχει ειδικό φίλτρο αναζήτησης ελεύθερου κειμένου, search_bar. Πραγματοποιεί αναζήτηση ελεύθερου κειμένου σε τίτλο, περιγραφή και κατηγορία. πχ GET /events/search?search_bar=music θα επιστρέψει εκδηλώσεις που είτε έχουν “music” στον τίτλο ή στη περιγραφή τους (source: https://stackoverflow.com/questions/62877278/django-filter-searching-on-multiple-fields-and-models).

Φίλτρα ανά κατηγορία

αναζήτηση με βάση μία ή περισσότερες κατηγορίες

Φίλτρα Εύρους Τιμών

είτε min_price, δηλαδή αναζήτηση εκδηλώσεων με τιμή κανονικού εισιτηρίου τουλάχιστον min_price,
είτε max_price, δηλαδή αναζήτηση εκδηλώσεων με τιμή κανονικού εισιτηρίου έως max_price.

Φίλτρα Χρονικού Διαστήματος

from_date, δηλαδή εκδηλώσεις από μια συγκεκριμένη ημερομηνία και μετά και
to_date, δηλαδή εκδηλώσεις που ολοκληρώνονται πριν από μία συγκεκριμένη ημερομηνία.

Φίλτρα Τοποθεσίας

Αναζήτηση με βάση μια πόλη (ελεύθερο κείμενο). Ψάχνει στο πεδίο city να περιέχει τη συγκεκριμένη αναζήτηση, όχι ακριβής αναζήτηση.

Σημείωση: *Συνολικά ένας χρήστης μπορεί να συνδυάσει πολλαπλά κριτήρια αναζήτησης.*

Αυτόματη Ενημέρωση Κατάστασης Εκδηλώσεων

Όταν η ημερομηνία και η ώρα λήξης μιας εκδήλωσης παρέλθει, η κατάστασή της μεταβαίνει αυτόματα σε COMPLETED. Για την υλοποίηση χρησιμοποιήθηκε Django Management Command, το οποίο εκτελείται περιοδικά μέσω του cron του λειτουργικού συστήματος κάθε μέρα.

Εικόνες

Για το ανέβασμα και τη διαχείριση εικόνων χρησιμοποιήθηκε το ImageField του django, το οποίο χρειάζεται τη βιβλιοθήκη Pillow. Στη βάση αποθηκεύεται η διαδρομή κάθε εικόνας, ενώ οι ίδιες οι εικόνες για τα events αποθηκεύονται στον υποφάκελο events του media φακέλου που ορίστηκε στα settings. (source: https://dev.to/atifwattoo/how-to-create-an-image-model-in-django-with-settings-1le3).

Κατάσταση Εκδηλώσεων

- Μπορούν να διαγραφούν draft events ή published events χωρις κρατήσεις.
- Μόνο published events μπορούν να δεχτούν κρατήσεις.
- Μετά την ημερομηνία λήξης το event γίνεται COMPLETED.

Συντεταγμένες

Ο υπολογισμός των συντεταγμένων έγινε με συνδυασμό χώρας-πόλης-διεύθυνσης. Χρησιμοποιήθηκε η υπηρεσία Notimatim του OpenStreetMap (https://wiki.openstreetmap.org/wiki/Nominatim). Συγκεκριμένα χρησιμοποιείται geocoding

και επιστρέφονται οι συντεταγμένες. Στο αρχείο users/utils/geocoding.py, υπάρχει η συνάρτηση που επιστρέφει τις συντεταγμένες.

Χωρητικότητα

Για κάθε εκδήλωση υπάρχει το πεδίο capacity.

- Ο συνολικός αριθμός εισιτηρίων που είναι διαθέσιμα δεν θα πρέπει να υπερβαίνει αυτόν τον αριθμό.
- Κατά την δημιουργία ενός νέου τύπου εισιτηρίου υπολογίζεται ο ήδη υπάρχον αριθμός εισιτηρίων, προστίθεται ο καινούργιος και ελέγχεται ότι δεν υπερβαίνει το capacity.
- Κατά την επεξεργασία ήδη υπάρχοντος τύπου εισιτηρίου, συγκεκριμένα του πεδίου quantity γίνεται ξανά ο ίδιος έλεγχος με τη διαφορά ότι αυτή τη φορά ελέγχεται και ότι ο νέος αριθμός του συγκεκριμένου τύπου στο οποίο γίνεται η επεξεργασία δεν πρέπει να είναι μικρότερος από τον αριθμό των εισιτηρίων που έχουν ήδη πωληθεί (δηλαδή old_quantity-old_available > new_quantity).
- Επίσης κατα την επεξεργασία εκδήλωσης δεν επιτρέπεται η αλλαγή του capacity σε αριθμό μικρότερο από το σύνολο των quantity όλων των ticket types που ήδη υπάρχουν.
- Κατά τη δημιουργία μιας κράτησης, ελέγχεται αν υπάρχουν αρκετά διαθέσιμα εισιτήρια και μειώνεται το πεδίο available ανάλογα. Το συγκεκριμένο πεδίο ανανεώνεται αυτόματα και όταν αλλάξει το quantity για κάποιο τύπο εισιτηρίου.

Περιεχόμενο Κράτησης

Κάθε κράτηση περιέχει πολλά booking items. Αυτό σημαίνει ότι κάθε κράτηση μπορει να περιλαμβάνει έναν ή περισσότερους τύπους εισιτηρίων.

Pagination

Το pagination υπάρχει by default στα generic views που χρησιμοποιήσαμε. Έχει οριστεί στα settings page size = 10

Validators

Δημιουργήθηκαν validators για την σωστή εισαγωγή ΑΦΜ και κινητού τηλεφώνου και για τον έλεγχο ισχύος του κωδικού πρόσβασης.

Frontend

Ενιαία σελίδα δημιουργίας και επεξεργασίας εκδήλωσης

Και οι δύο λειτουργίες εμφανίζουν την ίδια φόρμα, με μόνη διαφορά ότι στην επεξεργασία τα πεδία είναι προσυμπληρωμένα. Η διάκριση γίνεται από την ύπαρξη αναγνωριστικού στη διαδρομή.

Προσωρινή αποθήκευση και δημοσίευση.

Ο διοργανωτής μπορεί να αποθηκεύσει μια εκδήλωση ως πρόχειρη με ελλιπή στοιχεία και να την ολοκληρώσει αργότερα. Ο έλεγχος πληρότητας εφαρμόζεται μόνο κατά τη δημοσίευση, τόσο στο frontend, ώστε ο χρήστης να ενημερώνεται άμεσα για το τι λείπει, όσο και στο backend.

Προκαθορισμένος παραλήπτης στα μηνύματα.

Ο κάθε χρήστης δεν μπορεί να στείλει μήνυμα σε όποιον θέλει. Ο διοργανωτής “στέλνει” αυτόματα σαν confirmation της κράτησης και μετά ο χρήστης μπορεί να απαντήσει σε αυτό το μήνυμα αν θέλει να επικοινωνήσει με τον διοργανωτή.

Ένδειξη νέων μηνυμάτων.

Ο αριθμός των αδιάβαστων μηνυμάτων εμφανίζεται στη μπάρα πλοήγησης και ανανεώνεται σε κάθε μετάβαση σελίδας.

Επαναχρησιμοποιούμενα components.

Στοιχεία που εμφανίζονται σε πολλές σελίδες, όπως τα κουμπιά, οι κάρτες εκδηλώσεων και τα breadcrumbs, υλοποιήθηκαν ως ξεχωριστά components που δέχονται τα δεδομένα ως props.

Recommendation System

Οργάνωση Κώδικα

Η λογική του recommendation system βρίσκεται στον φάκελο recommendation. Ειδικότερα:

- Τα αρχεία data.py , evaluate.py, train.py, recommend.py αφορούν την υλοποίηση του αλγορίθμου με δεδομένα από το dataset.
- Τα αρχεία με πρόθεμα database_ αφορούν τα πραγματικά αρχεία κώδικα που χρησιμοποιούνται στο app για τις προτάσεις.
- Το model.py είναι η περιγραφή του μοντέλου και είναι κοινό.

Σχετικά με το dataset, για την προβολή των RMSE train και test set ανά epoch, το Precision@10, το Recall@10 και τον αριθμό των χρηστών που συμμετείχαν στην αξιολόγηση αρκεί η εντολή

```
python recommendation/recommend.py
```

μέσα στον φάκελο backend.

Offline Dataset

Χρησιμοποιήθηκαν τα αρχεία του dataset: events, users, event_interest. Το interested αφορά θετική προτίμηση και το not interested αρνητική. Συγκεκριμένα δόθηκε rating 1 για interested και 0 για not interested. Αν υπάρχουν πολλές εγγραφές για το ίδιο ζεύγος (user,event) κρατάμε το μέγιστο rating.

Δεδομένα Βάσης

- Για την πραγματική αξιολόγηση events για κάθε χρήστη έγινε χρήση του ιστορικού επισκέψεων και κρατήσεων.
- Συγκεκριμένα δόθηκε rating 1 για κράτηση σε κάποιο event και 0.3 για επίσκεψη της σελίδας ενός event (σε περιπτωση που υπάρχουν δεδομένα για τα 2 για ένα event κρατάμε το μέγιστο). Θεωρήσαμε δηλαδή την κράτηση σημαντικά πιο ισχυρή από απλή αλληλεπίδραση.
- Επίσης από τις προτάσεις αφαιρέθηκαν τα events που υπήρχαν στο σύνολο των αλληλεπιδράσεων για οποιοδήποτε λόγο (είτε λόγω κράτησης, είτε λόγω απλής επίσκεψης). Μάλιστα χρησιμοποιούνται events με ημερομηνία και ώρα έναρξης μετά την τρέχουσα και που είναι PUBLISHED.

Evaluation με βάση metrics

Το evaluation έγινε offline με δεδομένα από το dataset, ώστε να αξιολογηθεί το μοντέλο με βάση κάποιες μετρικές και έπειτα γνωρίζοντας πως λειτουργεί ορθά να μπορεί να επεκταθεί και στα δεδομένα της βάσης και των πραγματικών χρηστών. Οι μετρικές ήταν οι εξής:

- Precision@10: από τις 10 προτάσεις που έδωσα στον χρήστη, πόσες ήταν πραγματικά θετικές για αυτόν;
- Recall@10: από όλα τα πραγματικά θετικά events που είχε ο χρήστης στο test set, πόσα κατάφερε το μοντέλο να βάλει στις top 10 προτάσεις;

Κρατήσαμε τα πραγματικά θετικά events τα οποία κρύφτηκαν στο test set και για τα οποία το μοντέλο έχει κάποια δεδομένα/πληροφορίες, δηλαδή εμφανίστηκαν στα train events (όχι απαραίτητα για τον συγκεκριμένο χρήστη που ελέγχω κάθε φορά).

Ο Biased Matrix Factorization αξιολογήθηκε με train/test split ανά χρήστη.

- Ως relevant items ορίστηκαν εκδηλώσεις με interested=1. Η αξιολόγηση σε **369 χρήστες** έδωσε **Precision@10=0.0312** και **Recall@10=0.2967** (αποτελέσματα από την κλήση της συνάρτησης evaluate στο αρχείο backend/recommendation/evaluate.py). Επομένως, ανακτάται μέρος των κρυμμένων θετικών προτιμήσεων στις κορυφαίες προτάσεις.
- Για κάθε χρήστη γνωρίζουμε στο test set περίπου 1 θετικό event άρα το Precision@10 είναι λογικό να είναι αισθητά μικρό.
- Επειδή θέλουμε relevant events να εμφανίζονται όσο το δυνατόν ψηλότερα στην κατάταξη, προτιμήσαμε να εισάγουμε τιμές για το λ (περιορισμός του πόσο μεγάλες μπορούν να γίνουν οι παράμετροι), το η (learning rate) και τον αριθμό των epoch με

τέτοιο τρόπο ώστε να αυξηθεί το Recall@10 και Precision@10 και δευτερευόντως να ελαχιστοποιηθεί το RMSE.

Training, Saved Model & Online Recommendation

Δημιουργήθηκε management command το οποίο θα τρέχει ως cron job ανα 30 λεπτά. Μετά από κάθε train το μοντέλο αποθηκεύεται με τη χρήση της pickle επομένως κάθε φορά στο recommendation endpoint χρησιμοποιείται το πιο πρόσφατα αποθηκευμένο μοντέλο.

Σχεδιαστικές Επιλογές

- Δεν χρησιμοποιήθηκαν απευθείας τα πραγματικά ids των χρηστών και των εκδηλώσεων, αλλά πριν τη δημιουργία του πίνακα των αλληλεπιδράσεων έγινε map των πραγματικών ids σε indices. Αυτό γίνεται επειδή το μοντέλο αποθηκεύει τα latent factors και τα biases σε πίνακες, όπου κάθε θέση αντιστοιχεί σε έναν χρήστη ή event. Προφανώς οι πίνακες αυτοί χρειάζονται συνεχόμενες θέσεις.
- Το μοντέλο παράγει προσωποποιημένες προτάσεις για events που διαθέτουν learned latent representation. Events χωρίς προηγούμενες αλληλεπιδράσεις δεν περιλαμβάνονται στα events που πιθανόν να προτείνει ο αλγόριθμος και συνεπώς αντιμετωπίζονται ως item cold-start περιπτώσεις.

Επίλογος

Η ανάπτυξη της εφαρμογής ξεκίνησε με το backend κάνοντας ένα ένα κάθε app. Κατά την ανάπτυξη του backend ιδιαίτερα απαιτητική φάνηκε η διαχείριση των φίλτρων για την αναζήτηση, καθώς και ο τρόπος αποθήκευσης των εικόνων. Τη λύση έδωσε το DRF, γι αυτό και για τις περισσότερες λειτουργίες χρησιμοποιήθηκαν built-in λειτουργίες, μειώνοντας σημαντικά το μέγεθος κάθε αρχείου κώδικα και καθιστώντας την εύρεση λαθών πιο εύκολη από το αναμενόμενο. Σημαντική πρόκληση ήταν και η διαχείριση των αρχείων του dataset για τον αλγόριθμο συστάσεων αλλά και η διαδικασία αξιολόγησής του. Σε πιο ειδικές δυσκολίες, η δημιουργία event ήταν απαιτητική μιας και έπρεπε να υποστηρίζει και προσωρινή αποθήκευση. Επομένως, στήθηκε μια αλυσίδα 3+ endpoint που θα επρεπε να καλούνται διαδοχικά. Οπότε δημιουργείται αρχικά το event σαν draft σε κάθε περίπτωση και εν τέλει αν χρειαστεί κανονική δημοσίευση καλείται απευθείας και το endpoint για το publish (βλ. ενότητα Endpoints). Πολλές δυσκολίες εμφανίστηκαν εκεί που έπρεπε να συνεργαστούν το frontend με το backend. Για παράδειγμα, όταν προσθέσαμε τη δυνατότητα της προσωρινής αποθήκευσης, οι έλεγχοι του backend έσκαγαν επειδή περίμεναν συμπληρωμένα πεδία. Παρόμοια, χρειάστηκε προσοχή ώστε τα εισιτήρια να μη δημιουργούνται δύο φορές κάθε φορά που ο διοργανωτής αποθήκευε τις αλλαγές του. Βασικό ερώτημα ήταν και αν χρειαζόντουσαν διάφοροι ελεγχοι στις φόρμες και στο backend και στο frontend και τελικά χρησιμοποιήθηκαν έλεγχοι και στα δυο. Στο frontend ήταν κυρίως για την καλύτερη εμπειρία χρήστη και για ανατροφοδότηση στον χρήστη σε περίπτωση σφάλματος, ενώ ο πραγματικός έλεγχος γίνεται στο backend.

Τέλος, ορισμένα προβλήματα ήταν δύσκολο να εντοπιστούν, γιατί δεν παρήγαγαν σφάλμα αλλά κάποια πιθανόν λάθος συμπεριφορα, όπως φίλτρα που αγνοούνταν σιωπηλά ή πεδία ημερομηνίας που εμφανίζονταν κενά. Σε αυτές τις περιπτώσεις χρειάστηκε παρακολούθηση των αιτημάτων μέσα από τα εργαλεία του browser.